

Context is *compiled*, not collected.

A **context management system** is the runtime machinery that turns all *potentially available information* into the **smallest sufficient working set** for one particular model invocation.

authorize

retrieve

rank

transform

compress

assemble

...then persist & repeat

Storage problem? No — an *optimization* problem.

The naive mental model treats the context window as a bucket: keep appending messages, files, and tool results until it overflows. The system described here replaces that with an explicit objective.

BEFORE · ACCUMULATE

Everything that ever entered the conversation stays visible. The window fills with:

- ✗ irrelevant tokens
- ✗ latency and cost
- ✗ stale or contradictory information
- ✗ information leakage
- ✗ prompt-injection exposure
- ✗ loss of important state when it finally overflows

AFTER · COMPILE

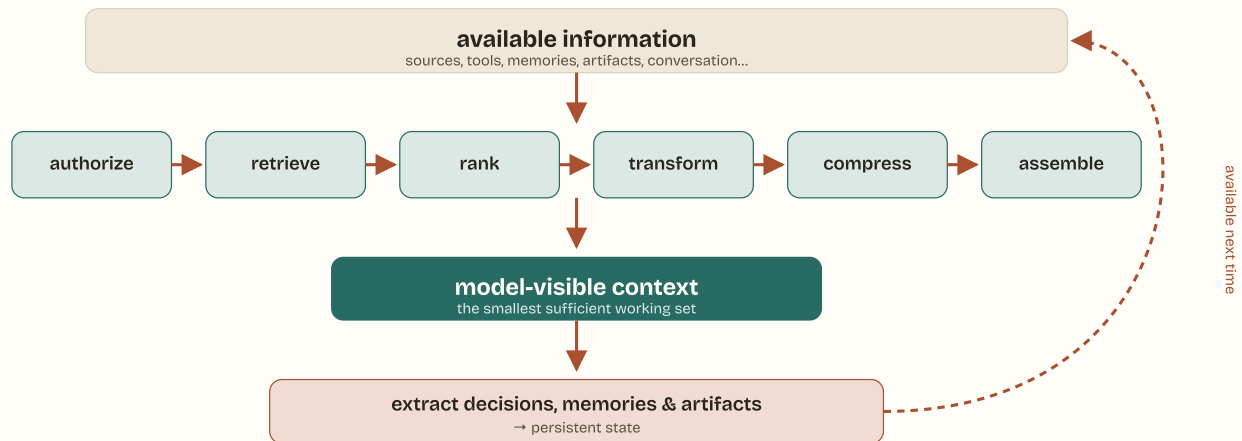
Each invocation gets a freshly built working set, chosen to:

- ✓ **maximize** expected task performance
- ✓ **minimize** every item on the left — simultaneously

Those six failure modes aren't accidents of long conversations; they are the exact quantities the system is built to drive down, on every single call.

Six stages — and a loop back

The core architecture is a pipeline from *available information* to *model-visible context*. Crucially, it doesn't end at the model: what the invocation produces is extracted into persistent state, which becomes available information for the **next** compile pass.



1 **authorize** — determine what this user, agent, and task are allowed to access.

3 **rank** — estimate which pieces are useful for the *present step*.

5 **compress** — trim, summarize, deduplicate, or defer to fit token, cost, and latency budgets.

2 **retrieve** — locate candidate sources, connectors, tools, memories, artifacts.

4 **transform** — convert heterogeneous state into model-ready instructions, evidence, and messages.

6 **assemble** — build the final structured input for the chosen model.

The signature: *context_t*

The subscript t is the whole point — context is a **function evaluated at every step**, not a variable that grows. Eight inputs go into each evaluation:

```
contextt = compile(  
  current task, // what step t is trying to achieve  
  conversation state, // recent + summarized history  
  durable memory, // preferences, decisions, learned facts  
  available data, // docs, code, records, tickets  
  available tools, // APIs, skills, delegable agents  
  model characteristics, // the chosen model shapes the format  
  token budget, // hard ceiling on cost & latency  
  security policy // what may be seen at all  
)
```

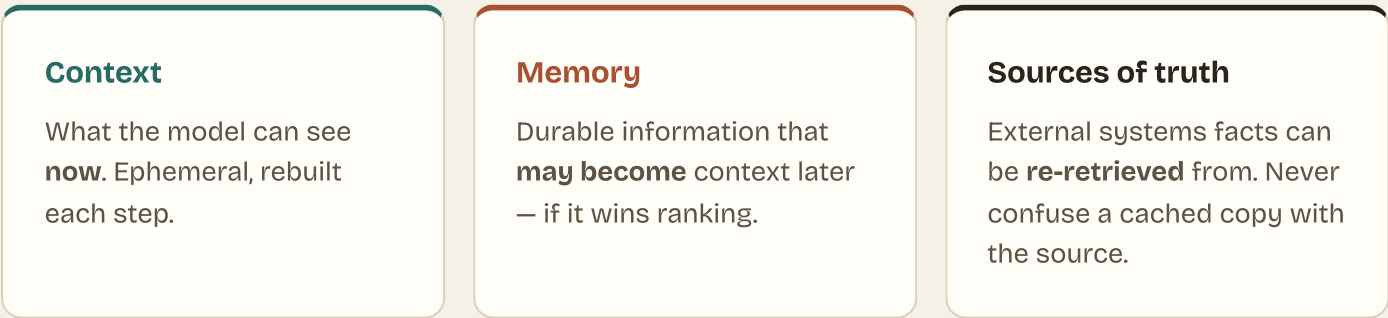
Note what's *parameters* here: the **model**, the **budget**, and the **security policy** are inputs to compilation — the same task compiles differently for a different model, a tighter budget, or a stricter policy.

Seven classes in, three places to live

The system juggles seven distinct classes of context — each with different lifetime, authority, and risk:



...and the distinction the system must never blur:



One compile pass, traced

An illustration of the doc's pipeline on a familiar scenario: a coding agent's next step is *"fix the failing test."* Watch each stage earn its keep.

authorize	This task may read the repo and its tickets — not another team's incident channel. Policy runs <i>before</i> retrieval, so forbidden data is never even a candidate.
retrieve	Pull candidates: the failing test file, the module under test, the ticket, a memory noting a past decision about this module.
rank	The stack trace and the module outrank the ticket's boilerplate for <i>this</i> step. Ranking is per-step, not per-conversation.
transform	Heterogeneous state becomes model-ready form: code as annotated snippets, the ticket as a three-line summary, tools as structured definitions.
compress	Older conversation turns collapse into a summary; duplicate log lines are deduplicated; low-rank candidates are deferred, not deleted — the budget holds.
assemble	Instructions, task state, evidence, and tool definitions are linked into the structured messages this specific model expects.
...persist	After the model runs: "chose approach B over A" is extracted as a durable decision. Next invocation, it's <i>available information</i> — the loop closes.

Four jobs in one system

The doc's compact description: *"a query planner, memory manager, policy engine and linker for model inputs."* Its nine responsibilities sort cleanly under those four hats:

Query planner

Discovery & ingestion · retrieval & ranking — find what could help and estimate what will.

Policy engine

Identity & policy enforcement · provenance & conflict handling — who may see what, where facts came from, which version wins.

Memory manager

Lifecycle management — what stays ephemeral, what becomes durable, what expires or must be revalidated.

Linker

Context compilation · budget management · handoff semantics — build the invocation, fit the budget, and pass a sub-agent only the state it needs, never the whole parent conversation.

Cross-cutting: observability & evaluation. Record what context was selected and measure whether the selection improved or degraded the outcome — the pipeline is tunable only if it's observable.

Keep the four terms straight

Context engineering — the discipline and design process.

Context management system — the runtime implementation.

Context compiler — the component that constructs one specific model invocation.

Memory system — the durable storage and recall subsystem.